Land Use Classification from Satellite Images Using Deep Learning

Yagna Saikumar

COMP-671 Deep Learning, Fall 2026 · Rivier University

Instructor: John Glossner

Code: `https://github.com/Rivier-Computer-Science/FA26-COMP-671-Yagna`

October 2026

Abstract

Knowing how land is used, where forests are shrinking, where cities are growing and where farmland lies, matters for environmental monitoring and planning, but checking large areas by hand is slow and expensive. This project builds a convolutional neural network (CNN) that assigns a 64×64 Sentinel-2 satellite image to one of ten land-use classes, using the public EuroSAT dataset of 27,000 labelled images. A non-deep baseline, logistic regression on downsampled pixels, reaches only 44.7% test accuracy (macro-F1 0.439). A four-block CNN trained from scratch reaches 98.0% (macro-F1 0.979), a gain of 53 points, which shows that learning spatial features is essential for this task. A controlled comparison isolates the effect of geometric data augmentation (flips and 90° rotations): over three random seeds, augmentation raises macro-F1 from 0.9727 ± 0.0020 to 0.9781 ± 0.0015 and wins in all three seeds, while the training-loss curves show that it mainly acts as a regularizer. Failure analysis shows that the remaining errors concentrate in visually similar agricultural classes (AnnualCrop, PermanentCrop, Pasture), which a single-date RGB image cannot reliably separate. The report closes with the limitations of the evaluation protocol and a discussion of responsible use.

1 Problem Statement and Motivation

Governments, environmental agencies and researchers need up-to-date maps of land use: to track deforestation, measure urban growth, plan infrastructure, and monitor agriculture. Satellites such as the European Space Agency's Sentinel-2 photograph the entire land surface of the Earth every few days and make the data freely available (Drusch et al., 2012). The bottleneck is no longer collecting images but interpreting them. Labelling land use by hand over a country-sized area is slow, expensive and hard to repeat often enough to track change.

Task. This project addresses a specific, well-defined version of the problem: *given a small satellite image patch, predict which of ten land-use classes it shows.* The classes are AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake.

- **Input:** one 64×64 pixel RGB image patch from Sentinel-2 (10 m per pixel, so each patch covers about 640×640 m on the ground).

- **Output:** one of the ten class labels, together with a confidence score from the softmax output.

A classifier like this is a building block for land-use mapping: a large scene is cut into patches, each patch is classified, and the predictions are stitched back into a map.

Research questions. The project is designed to answer two questions:

1. **Is deep learning needed?** Does a CNN meaningfully outperform a simple, non-deep baseline that only sees pixel colours?
2. **Does data augmentation help?** Holding everything else fixed, does adding geometric augmentation (flips and rotations) during training improve generalization to unseen images?

Success criteria. These were fixed before running the experiments. The primary metric is **macro-averaged F1 on a held-out test set**, reported alongside accuracy. A meaningful result requires (i) a CNN with at least 90% test accuracy, (ii) a margin of at least 15 points over the pixel baseline, and (iii) for augmentation to count as helpful, a macro-F1 gain of more than about 0.5 points that holds consistently across random seeds.

Scope. The project is deliberately limited to the ten EuroSAT classes, RGB bands only, and a compact CNN trained from scratch. This makes the full pipeline, including repeated training runs, feasible on a free cloud GPU within a semester, while still allowing a rigorous comparison.

2 Data Description

Source. The project uses **EuroSAT** (Helber et al., 2019), a public benchmark built from Sentinel-2 imagery covering 34 European countries. The RGB version contains 27,000 labelled 64×64 images in ten classes. It is loaded directly with `torchvision.datasets.EuroSAT`, which downloads it automatically (about 94 MB); no manual steps are required.

Class balance. Figure 1 shows the number of images per class. The dataset is roughly balanced: each class has between 2,000 (Pasture) and 3,000 images. Because the classes are not perfectly equal, **macro-F1**, which weights every class equally, is used as the primary metric rather than accuracy alone, so that the smaller classes count as much as the larger ones.

Visual character. Figure 2 shows random examples from each class. Some classes have a distinctive appearance (dark, uniform water for SeaLake; dense rooftop grids for Residential; a single linear feature for Highway and River). Others, especially the three agricultural classes, are all green or brown fields with similar textures. This already suggests where a model is likely to struggle.

Splits. The data comes as a single pool with no official split, so a split was created and saved:

- **70% train / 15% validation / 15% test** (18,900 / 4,050 / 4,050 images), **stratified by class** so that every split has the same class proportions.
- The split is generated with a fixed seed (42), and the image indices are saved to `results/splits.json`. Every experiment in this report uses exactly the same split.

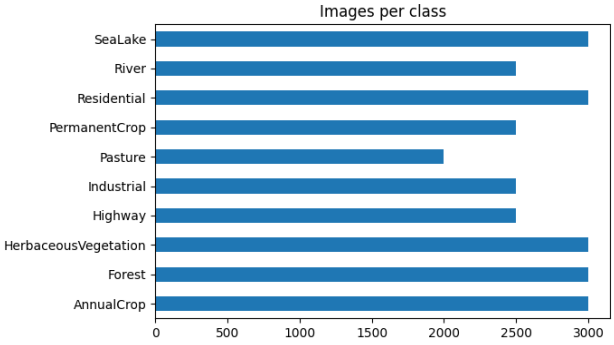


Figure 1: Number of images per class in EuroSAT. The classes are roughly balanced (2,000–3,000 each); Pasture is the smallest class.



Figure 2: Six random training images per class. Water, forest and residential patches look distinctive, while AnnualCrop, PermanentCrop, Pasture and HerbaceousVegetation share similar colours and field textures.

Table 1: CNN architecture (1,176,746 trainable parameters). Each block halves the spatial size.

Stage	Layers	Output size
Input	RGB image	$3 \times 64 \times 64$
Block 1	[Conv $3 \times 3 \to$ BatchNorm $\to$ ReLU] $\times 2$, MaxPool	$32 \times 32 \times 32$
Block 2	same, 64 channels	$64 \times 16 \times 16$
Block 3	same, 128 channels	$128 \times 8 \times 8$
Block 4	same, 256 channels	$256 \times 4 \times 4$
Head	Global average pooling $\to$ Dropout(0.3) $\to$ Linear	10 logits

- The validation set is used for model selection (choosing the best epoch); the test set is used *only once per model*, for the final numbers.

Preprocessing. Images are converted to tensors in $[0, 1]$ and normalized per channel. The normalization mean and standard deviation (mean $\approx [0.346, 0.382, 0.409]$, std $\approx [0.203, 0.137, 0.116]$) are computed **on the training split only**, so that no information from the test set leaks into preprocessing.

3 Method and Model Design

3.1 Baseline: logistic regression on pixels

A reasonable practitioner would first ask whether a simple model is enough. The baseline downsamples each image to 16×16 pixels and flattens it into a 768-dimensional vector ($16 \times 16 \times 3$), standardizes each feature, and fits a multinomial logistic regression (scikit-learn, Pedregosa et al., 2011). This model can use the *colour* at each position but has no notion of shape, texture or spatial pattern, and it is not translation-invariant. If it performed well, a deep model would not be justified. Its gap to the CNN measures how much spatial feature learning contributes.

3.2 CNN architecture

Because the input is an image, the main model is a convolutional neural network (Prince, 2023, Ch. 10). Convolutions share weights across positions, so a learned detector for, say, a road edge or a row of crops works anywhere in the image. Stacking layers lets the network combine simple edges into larger textures and structures. Table 1 summarizes the design.

The design choices and their justification:

- **Four conv blocks, channels doubling (32$\to$256).** Four poolings reduce 64×64 to 4×4, so the final features summarize large regions of the patch while depth stays modest for a small dataset.
- **Batch normalization** (Ioffe and Szegedy, 2015) after each convolution stabilizes and speeds up training (Prince, 2023, Ch. 11).
- **Global average pooling** (Lin et al., 2014) instead of large fully connected layers keeps the parameter count low (1.18 M), which reduces overfitting. It also fits the task: land-use class is a property of the whole patch rather than one location.

- **Dropout (0.3)** and **weight decay (10^{-4})** add regularization (Srivastava et al., 2014); (Prince, 2023, Ch. 9).

A model this size was chosen deliberately instead of a large pretrained network (e.g. a ResNet). It is enough for 64×64 inputs, trains from scratch in about seven minutes, and makes it practical to repeat the controlled comparison over several seeds.

3.3 Loss and optimization

The model is trained with **cross-entropy loss**, the standard choice for multiclass classification (Prince, 2023, Ch. 5). No class weighting is used because the classes are roughly balanced. Optimization uses **Adam** (Kingma and Ba, 2015) with learning rate 10^{-3}, batch size 128 and a **cosine learning-rate schedule** (Loshchilov and Hutter, 2017) over 25 epochs, which lets the model take large steps early and settle into a good minimum at the end (Prince, 2023, Ch. 6).

3.4 Controlled comparison: data augmentation

The ablation tests one design choice: **geometric data augmentation** (Shorten and Khoshgoftaar, 2019); (Prince, 2023, Ch. 9). During training, each image is randomly flipped horizontally and vertically and rotated by a random multiple of 90°. These transformations suit satellite imagery particularly well: an overhead image has no "up", so a rotated forest is still a forest, and the label never changes. Augmentation therefore shows the model new but valid versions of each image, which should discourage it from memorizing specific training images.

To make the comparison controlled, the two models, **CNN_plain** (no augmentation) and **CNN_aug** (augmentation), share *everything else*: the data split, architecture, initialization seed, optimizer, learning rate, schedule, number of epochs and model-selection rule. Augmentation is applied to the training set only; validation and test images are never augmented.

4 Experimental Setup

Protocol. Each CNN is trained for 25 epochs. After every epoch, macro-F1 is computed on the validation set, and the checkpoint with the best validation macro-F1 is saved. After training, that best checkpoint is evaluated once on the held-out test set. The baseline is fitted on the training set and evaluated on the same test set.

Metrics.

- **Macro-F1** (primary): the unweighted mean of per-class F1 scores, so every class counts equally.
- **Accuracy**: the fraction of test images classified correctly, for comparability with published results.
- **Confusion matrix** and **per-class precision, recall and F1**: to see *which* classes are confused.
- **Prediction confidence** (maximum softmax probability): to check whether the model knows when it is likely to be wrong.

Table 2: Training configuration, shared by both CNNs.

Setting	Value	Setting	Value
Split	70/15/15 stratified	Optimizer	Adam
Split seed	42	Learning rate	10^{-3}, cosine schedule
Batch size	128	Weight decay	10^{-4}
Epochs	25	Dropout	0.3
Loss	Cross-entropy	Model selection	Best validation macro-F1
Augmentation	H/V flip + 90° rotation	Seeds (robustness)	42, 1, 2

Table 3: Test-set results (4,050 images, seed 42). The CNN beats the pixel baseline by more than 50 points.

Model	Test accuracy	Test macro-F1
Baseline (logistic regression on pixels)	44.7%	0.439
CNN_plain (no augmentation)	97.3%	0.972
CNN_aug (flips + 90° rotations)	**98.0%**	**0.979**

Robustness over seeds. A single training run is affected by randomness in weight initialization and batch order. With a gap between models of under one point, one run cannot show whether a difference is real or noise. Both CNNs were therefore trained with three seeds (42, 1 and 2), and test results are reported as mean $\pm$ standard deviation, together with the per-seed gain.

Hyperparameters and compute. Table 2 lists the configuration, which is also saved to `results/config.json`. All training ran on a Google Colab NVIDIA T4 GPU: each CNN takes about 6.5–7 minutes and the baseline about 3 minutes, so the full notebook including the three-seed check runs in under an hour.

Reproducibility. The repository contains the single notebook implementing the whole pipeline, a `requirements.txt`, a README with run instructions, and all result figures and CSV files. Fixed seeds and the saved split recreate the same setup; small run-to-run differences of around ± 0.2 points are expected on a GPU, and the three-seed check (Section 5.3) measures exactly this spread.

5 Results

5.1 CNN versus baseline

Table 3 gives the main results on the held-out test set for the seed-42 run.

The CNN clearly meets all three pre-set criteria: it exceeds 90% accuracy, beats the baseline by 53 points (against a target of 15), and its 98.0% accuracy is in line with the roughly 98% that Helber et al. (2019) report for CNNs on the RGB version of EuroSAT. This agreement with published results is a useful sanity check that the pipeline has no serious bugs.

The confusion matrices in Figure 3 show *why* the baseline fails. It confuses classes that have similar **colours**, regardless of their structure. Forest and SeaLake, both dark, are mixed up in both directions (24% of Forest images predicted as SeaLake, 28% of SeaLake as Forest), and 38%

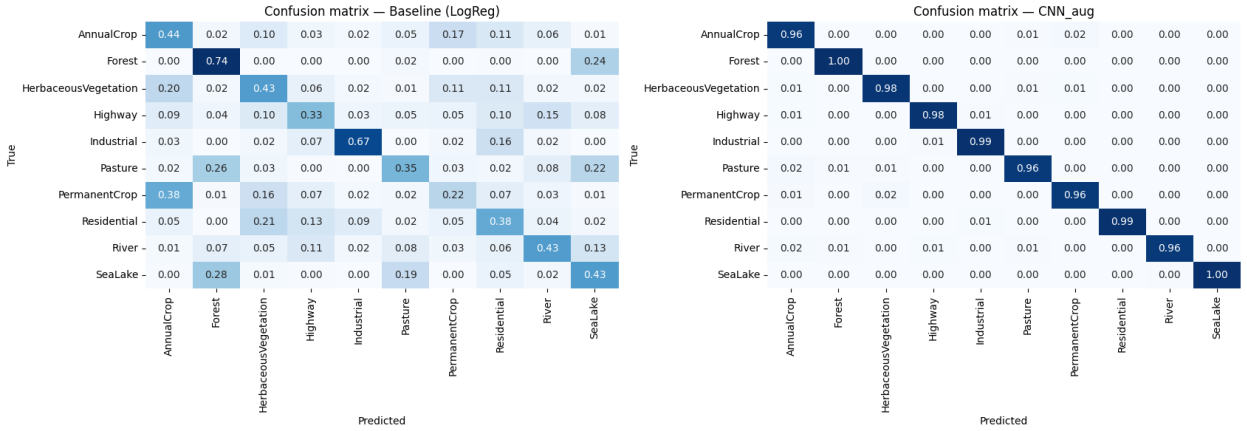


Figure 3: Row-normalized test confusion matrices for the baseline (left) and the best CNN (right). The baseline confuses classes with similar colours (Forest/SeaLake, PermanentCrop/AnnualCrop); the CNN is close to diagonal, with remaining errors among the agricultural classes.

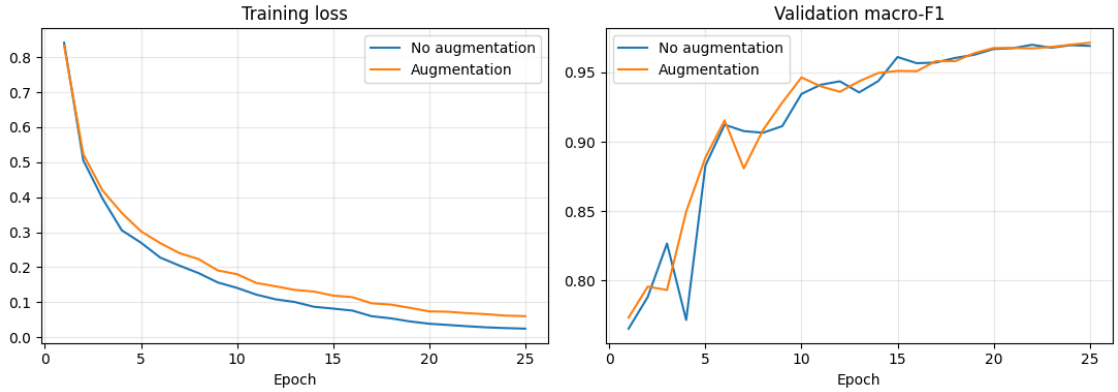


Figure 4: Training loss (left) and validation macro-F1 (right) per epoch for the two CNNs (seed 42). Validation F1 is nearly identical, but the model without augmentation reaches a much lower training loss, indicating stronger memorization of the training set.

of PermanentCrop images are predicted as AnnualCrop. The CNN's matrix is almost perfectly diagonal: every class reaches at least 96% recall. The difference between the two models is the ability to use texture and shape: the regular grid of a residential area, the straight line of a highway, the smooth surface of a lake.

5.2 Controlled comparison: effect of augmentation

In the seed-42 run, augmentation raised test macro-F1 from 0.972 to 0.979. Figure 4 shows the training dynamics behind this difference. Both models reach almost the same *validation* macro-F1 (about 0.97), but their *training* losses diverge: by epoch 25, the model without augmentation has driven its training loss down to 0.024, while the augmented model stays at 0.060. The plain model fits, and partly memorizes, its training images much more closely without gaining validation performance. Augmentation keeps the training task harder, because the model never sees exactly the same image twice, and this is the behaviour expected of a regularizer. Both validation curves are noisy in the first ten epochs, when the learning rate is high, and settle as the cosine schedule lowers it.

Table 4: Test macro-F1 for each seed and mean ± standard deviation over three seeds. Augmentation wins in every seed.

Seed	CNN_plain	CNN_aug	Gain (points)
42	0.9718	0.9792	+0.74
1	0.9751	0.9786	+0.36
2	0.9714	0.9764	+0.50
Mean ± std	0.9727 ± 0.0020	$\mathbf{0.9781 \pm 0.0015}$	**+0.53**

Table 5: Left: per-class test F1 of CNN_aug (seed 42), worst first. Right: the five most frequent confusions (counts out of 4,050 test images).

Class	F1
AnnualCrop	0.954
PermanentCrop	0.961
Pasture	0.968
HerbaceousVegetation	0.974
River	0.976
Highway	0.983
Industrial	0.988
Forest	0.994
Residential	0.996
SeaLake	0.998

True class	Predicted as	Count
AnnualCrop	PermanentCrop	9
PermanentCrop	HerbaceousVegetation	8
River	AnnualCrop	7
Pasture	AnnualCrop	7
PermanentCrop	AnnualCrop	5

5.3 Robustness across seeds

Because the difference between the two models is under one point, a single run is not enough evidence. Table 4 reports the three-seed experiment.

Augmentation improves macro-F1 in **all three seeds**, by +0.36 to +0.74 points, with a mean gain of **+0.53 points**. In accuracy terms, the means are $97.36 \pm 0.19\%$ without augmentation and $97.87 \pm 0.13\%$ with it. The gain is about the same size as the run-to-run spread of a single model, so it would be easy to miss or overstate from one run, but its sign is consistent. The augmented model also shows slightly smaller variation across seeds. The conclusion is that geometric augmentation gives a **small but consistent** improvement. It just meets the 0.5-point threshold set in advance, and its main measurable effect is reduced overfitting. With three seeds this is good evidence of a consistent direction, but not a precise estimate of the effect size.

6 Failure Analysis and Limitations

6.1 Where the model fails

Even at 98% accuracy, the remaining errors are not random. Table 5 lists the per-class F1 scores of the best model, worst first, and the most common confusions.

Three patterns stand out:



Figure 5: Random misclassified test images with true (T) and predicted (P) labels. Most errors involve agricultural or mixed scenes that are ambiguous even to a human viewer.

1. **Agricultural classes are confused with each other.** The three lowest-F1 classes are AnnualCrop, PermanentCrop and Pasture, and four of the five most common confusions are among the vegetation classes. From above, at 10 m resolution, these are all fields of green or brown with regular row or patch textures. The real difference between them is often *temporal*: annual crops are planted and harvested within a season, while permanent crops such as orchards and vineyards stay year-round. A single image taken on one date cannot show this. Near-infrared bands, which respond strongly to plant health and type, are also missing from the RGB version used here. The model is therefore not failing to learn; the input lacks the information needed to separate these classes reliably.
2. **Thin features are lost in farmland.** River images are sometimes predicted as AnnualCrop (7 cases). A narrow river can occupy only a few pixels of a 64×64 patch surrounded by fields, so the farmland dominates the image. This shows a limit of patch-level labels: one label must describe the whole patch, even when the labelled feature is small.
3. **Distinctive textures are easy.** SeaLake, Residential and Forest are almost perfect (F1 ≥ 0.994). These classes have unique, uniform textures that the CNN detects easily. This supports the reading that the model has learned genuine texture and structure cues rather than colour alone, because the baseline confused Forest and SeaLake heavily (Figure 3).

Figure 5 shows a sample of misclassified test images. Many of them are genuinely ambiguous to a human eye as well: field patches that could plausibly be either crop type, or mixed scenes containing two land uses.

6.2 Does the model know when it is wrong?

Figure 6 compares the model's confidence (maximum softmax probability) on correct and incorrect predictions. Correct predictions are almost all made with confidence above 0.97. Wrong predictions are spread widely between 0.4 and 1.0. This is useful in practice: a simple rule such as "send predictions below 0.9 confidence to a human reviewer" would catch a large share of the errors while letting most correct predictions through automatically. However, some wrong predictions still have confidence above 0.9, so confidence cannot be fully trusted, and an operational system would need calibration and human review.

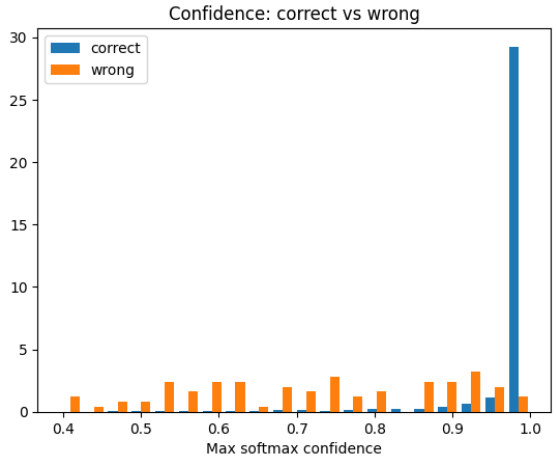


Figure 6: Distribution of maximum softmax confidence for correct (blue) and wrong (orange) test predictions, density-normalized. Correct predictions are concentrated near 1.0; wrong predictions are spread out, but some are still made with high confidence.

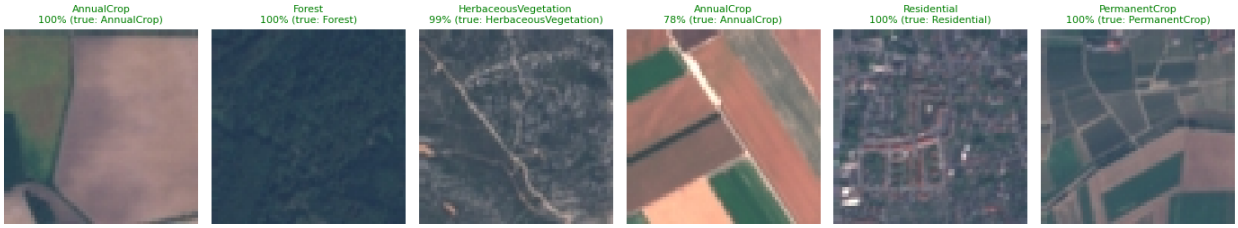


Figure 7: Inference demo: the saved model reloaded from disk classifies unseen test images. Titles show the predicted class, its confidence and the true class (green = correct, red = wrong).

6.3 Inference on unseen images

To confirm that the trained system works end to end, the saved weights are reloaded from disk and used to classify random test images, as a user of the model would. Figure 7 shows the predicted class and confidence for six such images.

6.4 Limitations of the study

- **Random split and spatial leakage.** The split is random at the image level. EuroSAT patches are cut from larger scenes, so neighbouring patches from the same area, with the same soil, season and lighting, can appear in both training and test sets. This likely makes the test score optimistic: performance on a completely new region could be noticeably lower. A spatially blocked split (Roberts et al., 2017) would give a more realistic estimate, but EuroSAT does not provide the location metadata needed to build one.
- **Geographic coverage.** All images come from Europe. Land use looks different elsewhere (field sizes, building materials, vegetation types), so these results say nothing about performance in Africa, Asia or the Americas.
- **RGB only.** Sentinel-2 records 13 spectral bands; this project uses 3. The bands left out, especially near-infrared, are exactly the ones most useful for separating vegetation types.
- **Fixed label set and single labels.** The model can only output ten classes and one label per

patch. Real scenes contain mixtures (a road through farmland) and land uses outside these ten classes (wetlands, mines, glaciers), which the model would force into the closest wrong class.

- **Limited seeds and one architecture.** The augmentation result rests on three seeds and one CNN design. The direction of the effect is consistent, but its exact size is uncertain, and it might be different for a larger or pretrained model.

7 Ethics, Limitations and Responsible Use

Land-use classification may look like a harmless technical task, but its outputs feed decisions with real consequences for people, so the risks need to be considered for this application specifically.

Consequences of errors. Land-use maps are used for zoning, agricultural subsidies, environmental enforcement and conservation planning. This model's weakest classes are crop types. If its outputs were used, for example, to check whether farmland is being used as declared for subsidy purposes, a confusion between AnnualCrop and PermanentCrop could wrongly flag a farmer, or let a misdeclaration pass. A 2–5% error rate on these classes means many affected parcels at a regional scale. The model's outputs should therefore support, not replace, human judgement, and decisions affecting individuals should require human review, for example by using low-confidence predictions as the trigger for that review (Figure 6).

Bias and fairness across regions. Because the model is trained only on European imagery, it is likely to be least accurate in places that look least like Europe: smallholder farms, informal settlements, tropical vegetation. If such a model were used to guide funding or environmental policy elsewhere, regions that are already under-represented in data and resources could receive the least accurate maps. Anyone deploying it outside Europe should first evaluate it on local labelled data.

Privacy and misuse. At 10 m per pixel, Sentinel-2 cannot show individual people, vehicles or the inside of private property, so the direct privacy risk is low. The same methods applied to high-resolution commercial imagery (under 1 m per pixel) raise very different concerns, including surveillance of private land and monitoring of specific communities. This project's conclusions should not be read as covering that setting.

Transparency. The test score of 98% is measured on a random split and is probably optimistic for new regions, as discussed above. Reporting it without that caveat could lead users to over-trust the model. Documentation of the model's intended use, training data and known weaknesses, in the spirit of model cards (Mitchell et al., 2019), should accompany any release.

Environmental cost. Training deep models consumes energy (Strubell et al., 2019). This project deliberately uses a small CNN (1.18 M parameters): the complete set of experiments, including all repeated runs, took under an hour on a single T4 GPU, a very small footprint. Larger models would cost more and are not clearly needed for this task, given that the small model already matches published accuracy.

8 Conclusion

This project built and evaluated a deep learning system that classifies Sentinel-2 satellite image patches into ten land-use classes, and it answered both of its research questions with evidence.

- **Deep learning is necessary for this task.** A compact CNN reaches 98.0% test accuracy (macro-F1 0.979), against 44.7% for a logistic-regression baseline on pixels. The baseline confuses classes of similar colour; the CNN separates them by learning texture and spatial structure.
- **Geometric augmentation helps a little, but consistently.** Over three seeds, flips and 90° rotations raise macro-F1 by +0.53 points on average and win in every seed. Their clearest effect is reduced overfitting: a training loss of 0.060 versus 0.024 at the same validation performance.
- **The remaining errors have a clear cause.** They concentrate in agricultural classes that look alike in a single RGB image. Separating them needs information the input does not contain: seasonal change over time and near-infrared light.

The most valuable next steps follow from the failure analysis: using all 13 Sentinel-2 bands, especially near-infrared, to separate crop types; using multi-date image sequences to capture seasonal change; testing transfer learning from a pretrained network such as ResNet-18; and, most importantly, evaluating on a geographically held-out region to measure how well the model generalizes to places it has never seen.

References

Drusch, M., Del Bello, U., Carlier, S., Colin, O., Fernandez, V., Gascon, F., Hoersch, B., Isola, C., Laberinti, P., Martimort, P., Meygret, A., Spoto, F., Sy, O., Marchese, F., & Bargellini, P. (2012). Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote Sensing of Environment*, 120, 25–36.

Helber, P., Bischke, B., Dengel, A., & Borth, D. (2019). EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7), 2217–2226.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448–456.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

Lin, M., Chen, Q., & Yan, S. (2014). Network in network. In *International Conference on Learning Representations (ICLR)*.

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 220–229.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Prince, S. J. D. (2023). *Understanding Deep Learning*. MIT Press.

Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913–929.

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6, 60.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3645–3650.